

Documento de Arquitetura e Engenharia de Software (SRS / SAD)

Projeto: Sistema Integrado de Gestão Comercial e Automação (PDV / ERP / CRM)

Status: Fase 1 — Planejamento de Requisitos e Arquitetura do MVP

Banco de Dados Previsto: Supabase (PostgreSQL Nativo)

Autor: Engenharia de Software

1. Contextualização e Estudo de Viabilidade (Mercado & Métricas)

Para que o desenvolvimento de um software tenha sucesso no varejo de proximidade/bairro, a arquitetura deve responder diretamente às dores reais da operação diária, resolvendo problemas operacionais e financeiros cruciais.

1.1. Contexto Econômico (Fontes: IBGE, SEBRAE e FENACON)

- **Relevância do Setor:** Micro e pequenas empresas (MPEs) do comércio representam 70% dos empregos formais do setor comercial no Brasil e compõem mais de 30% das aberturas anuais de novos CNPJs.
- **Volume Varejista:** Dados da Pesquisa Mensal do Comércio (PMC/IBGE) mostram um volume expressivo de operações em bens de consumo diário (alimentos, farmácia, vestuário, utilidades domésticas), demandando respostas rápidas de PDV e emissão fiscal diária sem gargalos.
- **A Grande Dor (Inadimplência e Margens Exíguas):** Segundo o SEBRAE, a concessão de crédito informal ("fiado") e a falta de controle sobre taxas bancárias/maquininhas figuram entre as maiores causas de mortandade precoce de negócios de bairro.

1.2. Justificativa Técnica da Arquitetura

A proposta de um sistema integrado unifica PDV + Cobrança Automatizada via WhatsApp + Conciliação de Cartões + Fiscal One-Click, eliminando a necessidade de o microempresário manter múltiplos softwares pagos separadamente.

2. Visão Geral da Arquitetura do Sistema

Para suportar operações de balcão de alta velocidade e, simultaneamente, processar rotinas em segundo plano (como disparos de WhatsApp e webhooks), adotaremos um padrão de Monólito Modular orientado a Eventos, integrado à infraestrutura Serverless e Realtime do Supabase.

Stack Tecnológica Selecionada

- **Backend / API:** API REST / WebSockets construída em Node.js (TypeScript), Python (FastAPI) ou PHP (Laravel) para regras de negócio complexas.
- **Frontend (PDV/Web):** React.js ou Vue.js (SPAs leves para garantir velocidade de renderização no caixa).
- **Banco de Dados & Serverless:** Supabase (PostgreSQL nativo com Realtime, Row Level Security, Auth e Storage).
- **Filas & Workers:** Redis / Supabase Webhooks + Workers em segundo plano para cobranças de WhatsApp sem travar a API principal.

3. Matriz de Priorização do MVP (Produto Mínimo Viável)

Módulo do Sistema	Pilar	Relevância no MVP	Complexidade
01. Credíário & Cobrança WhatsApp	Financeiro	Fase 1 (MVP Core)	Média
02. Conciliação de Cartões	Financeiro	Fase 2	Alta
03. Controle Preditivo de Estoque	Estoque	Fase 1 (MVP Core)	Média
04. Sincronização Catálogo/Micro-PDV	Vendas/Estoque	Fase 1 (MVP Core)	Média
05. Logística de Motoboys (Kanban)	Logística	Fase 3	Média
06. Emissão Fiscal	Vendas	Fase 2	Alta

Módulo do Sistema	Pilar	Relevância no MVP	Complexidade
One-Click			
07. Cotação de Compras/Fornecedores	Suprimentos	Fase 3	Média
08. CRM e Fidelização	CRM	Fase 2	Baixa
09. Fechamento de Caixa Cego & Comissões	Vendas	Fase 1 (MVP Core)	Baixa
10. Dashboard Precificação & DRE	Financeiro	Fase 2	Média
11. Central Multicanal	Atendimento	Fase 3	Alta
12. Trocas e Vales-Crédito	Vendas	Fase 1 (MVP Core)	Baixa

4. Detalhamento do MVP — Fase 1 (Core do Sistema)

Módulo 01: Vendas, Caixa Cego e Trocas (PDV Core)

Requisitos Funcionais (RF)

- **RF-PDV-01:** Permite abertura e fechamento de caixa por turno.
- **RF-PDV-02 (Caixa Cego):** No fechamento, o operador informa os valores em espécie e comprovantes sem visualizar o saldo esperado pelo sistema. O sistema gera um relatório de quebra de caixa apenas para os administradores.
- **RF-PDV-03:** Registra vendas associando produtos, formas de pagamento e cliente (opcional).
- **RF-PDV-04:** Permite efetuar trocas. O item devolvido retorna ao estoque e gera um ValeCredito no CPF/WhatsApp do cliente.

Regras de Negócio (RN)

- **RN-PDV-01:** Não é permitido realizar vendas com o caixa em estado FECHADO.
- **RN-PDV-02:** Vendas utilizando o ValeCredito devem abater o valor total da venda; caso o valor seja menor, gera-se um saldo remanescente vinculado ao cliente.

Módulo 02: Crediário ("Fiado") e Cobrança Automática

Requisitos Funcionais (RF)

- **RF-CRE-01:** Permite realizar vendas na modalidade "Crediário", definindo número de parcelas, valor e datas de vencimento.
- **RF-CRE-02:** Envia disparos de lembretes preventivos via WhatsApp (ex: 3 dias antes do vencimento) com a chave Pix copia-e-cola.
- **RF-CRE-03:** Envia lembretes pós-vencimento configuráveis de forma cordial.
- **RF-CRE-04:** Registra a quitação total ou parcial de parcelas, atualizando o saldo devedor do cliente.

Regras de Negócio (RN)

- **RN-CRE-01:** Vendas no crediário só podem ser concluídas se o cliente possuir limite de crédito ativo e saldo suficiente disponível.
- **RN-CRE-02:** O envio de mensagens deve respeitar horários comerciais (entre 08h e 18h) para evitar problemas legais e bloqueios no WhatsApp.

Módulo 03: Sincronização de Estoque e Alertas de Validade

Requisitos Funcionais (RF)

- **RF-EST-01:** Sincroniza em tempo real a quantidade de produtos disponíveis entre o estoque físico do PDV e o catálogo digital via Supabase Realtime.
- **RF-EST-02:** Registra produtos por lotes e datas de validade.
- **RF-EST-03:** Dispara alertas no dashboard para produtos com vencimento nos próximos N dias (configurável, ex: 30 dias).

Regras de Negócio (RN)

- **RN-EST-01:** Ao atingir o estoque zero no PDV, o item deve ser imediatamente desativado ou sinalizado como "Indisponível" no catálogo online.

5. Modelagem de Banco de Dados (Supabase / PostgreSQL)

Esquema em SQL compatível com Supabase, utilizando extensão de UUID e habilitando subscrição de Realtime para sincronização de catálogo:

```
-- Habilita extensão para UUID
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- 1. Tabela de Clientes
CREATE TABLE public.clientes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nome VARCHAR(150) NOT NULL,
  cpf_cnpj VARCHAR(20) UNIQUE,
  whatsapp VARCHAR(20) NOT NULL,
  limite_credito DECIMAL(10,2) DEFAULT 0.00,
  saldo_devedor DECIMAL(10,2) DEFAULT 0.00,
  criado_em TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Tabela de Produtos
CREATE TABLE public.produtos (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  codigo_barras VARCHAR(50) UNIQUE,
  nome VARCHAR(150) NOT NULL,
  preco_custo DECIMAL(10,2) NOT NULL,
  preco_venda DECIMAL(10,2) NOT NULL,
  estoque_atual INT DEFAULT 0,
  estoque_minimo INT DEFAULT 5,
  ativo_catalogo_web BOOLEAN DEFAULT TRUE,
  atualizado_em TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Tabela de Caixas
CREATE TABLE public.caixas (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  operador_id UUID NOT NULL,
  data_abertura TIMESTAMPTZ DEFAULT NOW(),
  data_fechamento TIMESTAMPTZ,
  valor_inicial DECIMAL(10,2) NOT NULL,
  valor_declarado_especie DECIMAL(10,2),
  diferenca_quebra DECIMAL(10,2),
  status VARCHAR(20) DEFAULT 'ABERTO' CHECK (status IN ('ABERTO', 'FECHADO'))
);

-- 4. Tabela de Vendas
CREATE TABLE public.vendas (
```

```

    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    caixa_id UUID REFERENCES public.caixas(id),
    cliente_id UUID REFERENCES public.clientes(id),
    valor_total DECIMAL(10,2) NOT NULL,
    forma_pagamento VARCHAR(30) NOT NULL,
    criado_em TIMESTAMPTZ DEFAULT NOW()
);

-- 5. Parcelas de Crediário (Fiado)
CREATE TABLE public.parcelas_crediarario (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    venda_id UUID REFERENCES public.vendas(id) ON DELETE CASCADE,
    cliente_id UUID REFERENCES public.clientes(id),
    numero_parcela INT NOT NULL,
    valor DECIMAL(10,2) NOT NULL,
    data_vencimento DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'PENDENTE' CHECK (status IN ('PENDENTE', 'PAGO',
'ATRASADO')),
    data_pagamento DATE
);

-- Habilita o Realtime na tabela de produtos para o Catálogo Web no Supabase
ALTER PUBLICATION supabase_realtime ADD TABLE public.produtos;
```

6. Próximos Passos Sugeridos

1. Criação do projeto no Supabase e execução da migration SQL.
2. Elaboração das especificações das APIs de integração (WhatsApp / Pix).
3. Desenvolvimento dos protótipos de tela do caixa e dashboard.